GOAL
====
Write Python scripts that download a payer's medical policy documents that relate to prior authorization / precertification / notification requirements (procedure codes, drugs, provider-administered medications, etc.) and save them as PDFs.

Save every document under ./downloaded/<PayerName>/ (create the folders if they do not exist). Use readable, human-friendly file names derived from the document's link text/label, not the cryptic file name in the URL.

Unless told otherwise, download the MOST RECENT version of each distinct document. When the request is instead "all versions published in <year>", download every distinct version for that year (see "Historical versions" below).

HOW TO FIND THE DOCUMENTS
=========================
1. Start from the payer's official provider prior-authorization / precertification web page. Confirm the URL actually returns HTTP 200 before relying on it (many guessed URLs 404 or redirect). Follow redirects and use the final URL.

2. Send a realistic desktop browser User-Agent header (and Accept: */*). Some payer sites return 403 or block default Python/requests headers.

3. Scrape the page for links to PDF documents whose URL or link text looks like a prior-auth / precert / notification document. Useful keywords: precert, pre-cert, prior auth(orization), preauth, notification list, advance notification, PA requirements, prior authorization and notification list, summary of changes.

4. Many payer pages are JavaScript-rendered or embed the real document list as data (e.g. JSON "content fragments" with HTML-escaped markup, where '<' appears as \u003c, '>' as \u003e, ' ' as \u0026nbsp;, and quotes are backslash-escaped). Parse that embedded markup with regex/HTML parsing and unescape it. Do NOT assume a plain <a href> scrape is enough — first fetch the page and inspect where the PDF links actually live.

5. Some pages group many markets/states/plans together and label lists by section heading such as "Current ..." vs "Previous ...". When you only want the most recent documents, keep the links under the "Current" and "Summary of changes" headings and SKIP anything under a "Previous" heading. When a page lists many states/plans, filter to only the ones the request asks for (match on the state/plan name in the link text or URL).

6. When two documents share nearly identical link text (e.g. a "Medical" list and a "Medication" list both called "Prior Authorization and Notification List"), disambiguate using the nearest section heading (headings often contain words like "medication", "provider-administered", "medical") and include that category in the saved file name so files don't collide.

DOWNLOADING & SAVING
====================
7. Download each document and VALIDATE it is a real PDF before saving: the response body must start with the bytes b"%PDF" (and/or Content-Type application/pdf). Skip and report anything that isn't a PDF.

DOWNLOADING & SAVING
====================

7. Download each document and VALIDATE it is a real PDF before saving: the response body must start with the bytes b"%PDF" (and/or Content-Type application/pdf). Skip and report anything that isn't a PDF.

8. De-duplicate by content: compute a SHA-256 hash of each downloaded body and skip writing a file whose content was already saved (the same PDF is often linked from multiple places or URLs).

9. Build safe file names from the link label: strip characters that are invalid in file names, collapse whitespace, ensure a single .pdf extension. Include the effective date and the category when available.

10. Add a small hard-coded FALLBACK list of the currently-known document URLs (with labels) for each payer, and use it only if scraping returns nothing (so a future page-layout change degrades gracefully instead of failing).

HISTORICAL VERSIONS (only when "all versions in <year>" is requested)
===

11. Payers often revise a list at the SAME URL over the year. To get every version published in a given year, also query the Internet Archive (Wayback Machine CDX API) for that URL, collapse by digest, and download each distinct snapshot from that year (use the "id_" raw snapshot form, e.g. https://web.archive.org/web/<timestamp>id_/<original-url>).

ROBUSTNESS & OUTPUT
===================

12. Handle network/parse errors per-document so one failure doesn't abort the whole run; print a clear progress log (what was found, downloaded, skipped, and why) and a final summary listing the saved files.

13. Be polite: reuse a single requests.Session, set reasonable timeouts, and add a short delay between many archive requests.

14. Verify the script actually runs and produces valid, non-empty PDFs before considering it done.

TIP FOR DISCOVERING URLS
========================

Public search engines are often blocked or return challenge pages when scripted (Google/Bing/DuckDuckGo may 202/403). Prefer probing candidate official payer URLs directly (check status codes) and inspecting the real page HTML/embedded JSON to discover the true PDF asset URLs.

EXAMPLE (already implemented in this workspace)
===

- download_aetna_precert_policies.py -> Aetna, all 2026 versions via Wayback
- download_humanaHealthyHorizon_precert_policies.py
  -> Humana Healthy Horizons in Oklahoma (Medicaid), most recent Medical + Medication prior-authorization/notification lists and their summaries, scraped from Humana's provider prior-authorization-lists page.
- download_ManyPayer_precert_policies.py -> many payers by name, one at a time